TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 9 (Part 3): Embeddings, Vector Databases & RAG Systems

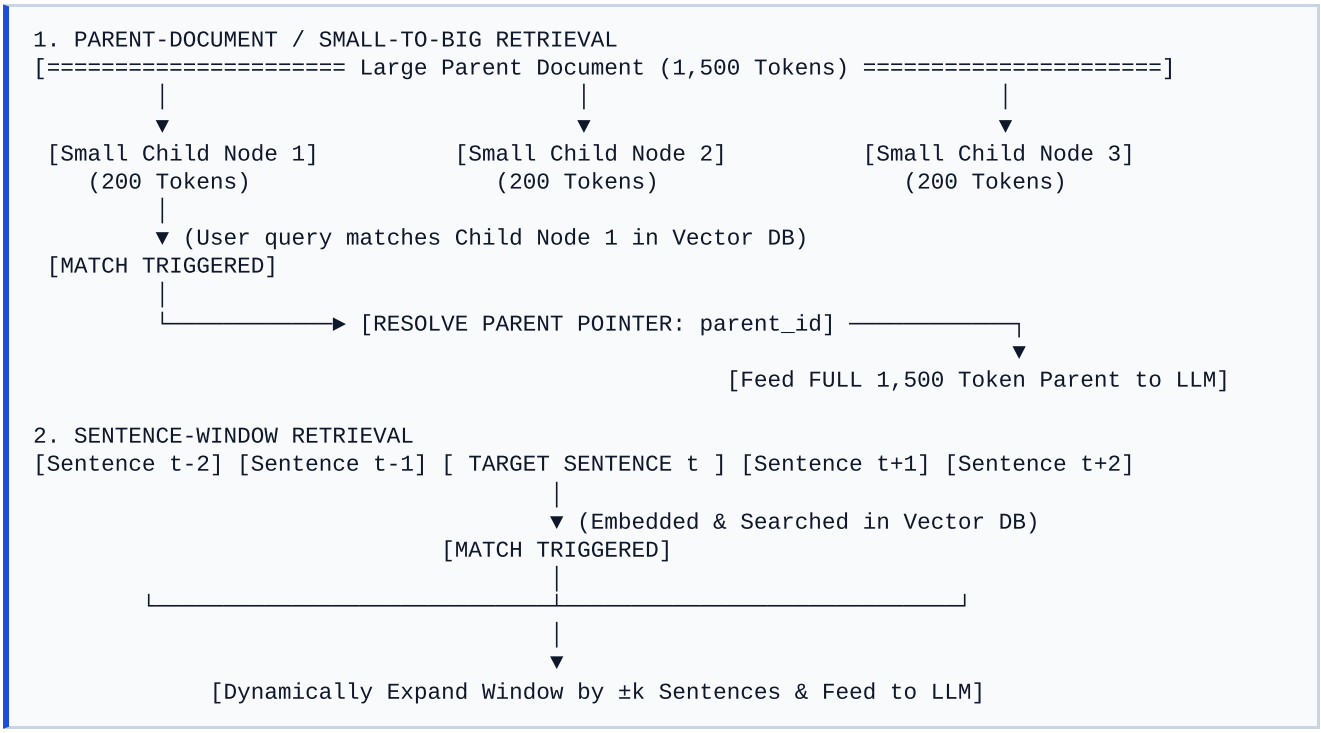
Module 9 Part 3 completes the deep dive into Retrieval-Augmented Generation, exploring multi-granular document representations, self-correcting agentic retrieval loops, knowledge graph integration, native multimodal ingestion, systematic architectural failure modes, and end-to-end evaluation engineering.

9.15 Hierarchical Retrieval: Parent-Document, Small-to-Big, and Sentence-Window Retrieval

Standard RAG architectures suffer from the **Embedding Granularity Conflict**:

- Small text fragments (e.g., 50–100 tokens) yield sharp, highly focused vector embeddings with minimal semantic dilution, but lack the contextual background required for an LLM to synthesize a complete response.
- Large text passages (e.g., 1,000–2,000 tokens) provide rich surrounding context to the LLM, but their vector representations become diluted averages of multiple topics, degrading bi-encoder retrieval accuracy.

Hierarchical retrieval resolves this tension by **decoupling the retrieval unit from the generation unit**: the index searches over fine-grained sub-chunks, but passes larger parent documents or expanded context windows to the generator.



1. Parent-Document (Small-to-Big) Architecture

- **Ingestion Mechanics:**
  1. The raw document is split into coarse **Parent Chunks** (typically 1,000–2,000 tokens). Each parent receives a unique UUID (`parent_id`).
  2. Each parent chunk is subdivided into multiple overlapping **Child Chunks** (typically 150–300 tokens).
  3. Every child chunk inherits its parent's metadata and explicitly stores `parent_id` in its payload.
  4. Only the **child chunks are embedded and indexed** in the vector database. The parent chunks are stored in an external key-value document store (Redis, DynamoDB, MongoDB, or disk-backed SQLite).
- **Query Phase:**
  1. The query matches the child chunk embeddings using standard ANN search.
  2. Rather than returning the child text, the system extracts the `parent_id` from the matched child nodes.
  3. The system deduplicates parent IDs (so multiple child hits from the same document do not duplicate text) and fetches the complete parent chunks from the key-value store to inject into the LLM context.

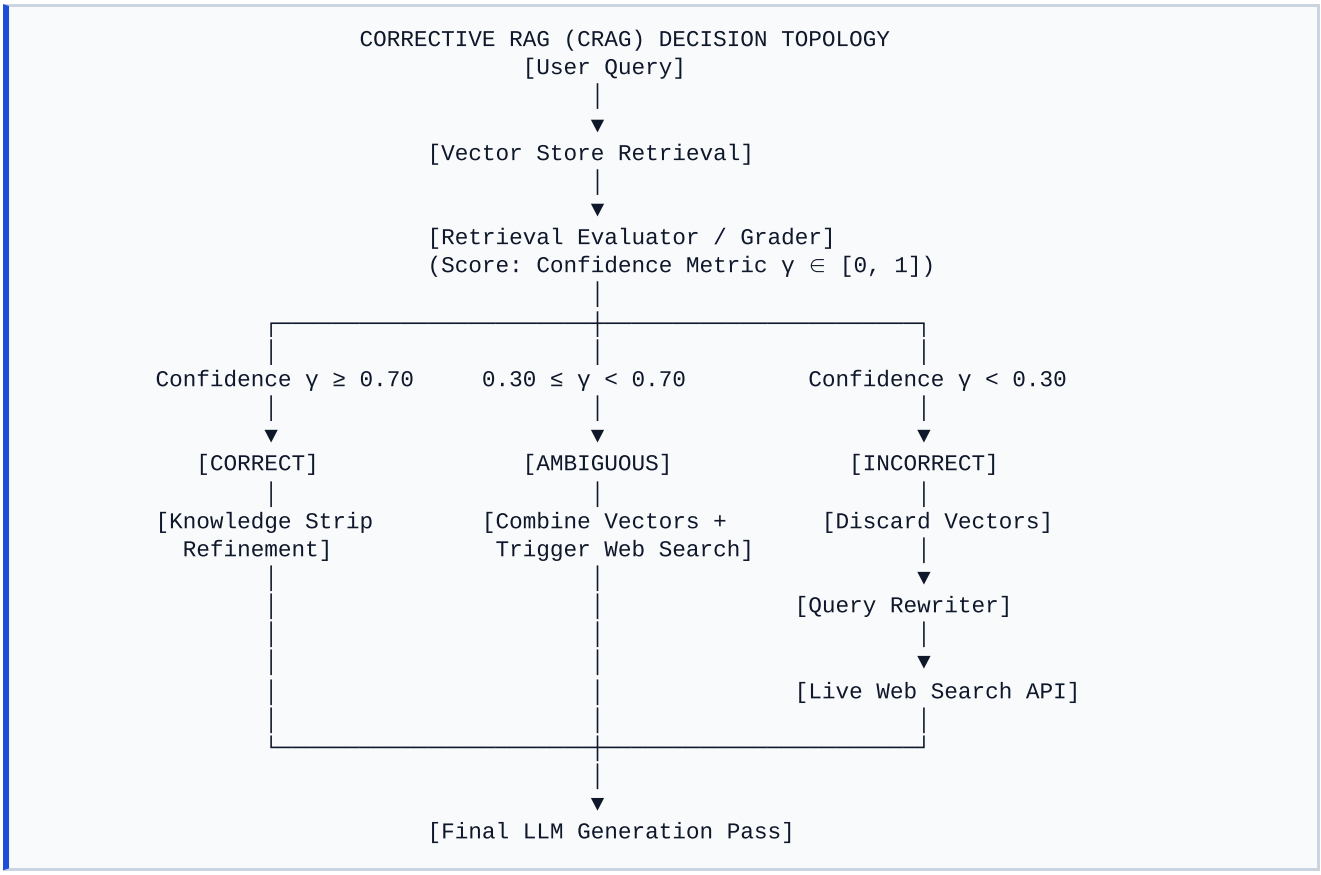
2. Sentence-Window Retrieval

Sentence-window retrieval isolates individual sentences for vector indexing while preserving surrounding linguistic context during generation:

- **Ingestion:** Text is parsed into discrete sentences using rule-based boundary detection (e.g., spaCy or NLTK). Each sentence is embedded individually.
- **Retrieval & Replacement:** When a sentence matches a query, the retrieval engine checks the sequence index and fetches a symmetric window of k sentences before and after ($[t-k, \ldots, t, \ldots, t+k]$, typically $k = 3$).
- **Advantage:** The embedding reflects a precise statement, but the LLM receives the adjacent sentences containing pronoun antecedents, caveats, and qualifications.

9.16 Quality-Aware Generation: Self-RAG and Corrective RAG (CRAG)

Static RAG pipelines execute a fixed sequence: *Retrieve* → *Augment* → *Generate*. If retrieval fails or returns irrelevant context, the LLM is forced to hallucinate or generate corrupted answers. **Quality-Aware RAG** introduces dynamic evaluation gates that audit retrieval relevance and factual consistency at inference time.



1. Corrective RAG (CRAG)

Introduced by Yan et al. (ICLR 2025), **Corrective RAG (CRAG)** implements an upstream evaluation step to grade the quality of retrieved documents before generation begins:

- **Confidence Scoring:** A lightweight evaluator (a fine-tuned classifier or an SLM like Claude 3.5 Haiku / GPT-4o-mini) assesses the relevance of retrieved documents to the query, assigning an aggregate confidence metric $\gamma \in [0, 1]$.
- **Three-Way Routing Topology:**
  1. **Correct ($\gamma \geq 0.70$):** Retrieval was successful. Chunks undergo **Knowledge Strip Refinement**—they are broken into granular sentence strips, irrelevant sentences are purged, and the remaining facts are recomposed into a concise context.
  2. **Incorrect ($\gamma < 0.30$):** Retrieval completely missed. Vector results are discarded to prevent polluting the prompt. The query is passed to an automated rewriter and dispatched to a live web search engine (Tavily, Brave, Google Search API) to retrieve external context.
  3. **Ambiguous ($0.30 \leq \gamma < 0.70$):** Vector retrieval was partially successful. The internal documents are merged with external web search results, filtered, and passed to generation.

2. Self-RAG (Self-Reflective Retrieval-Augmented Generation)

Introduced by Asai et al. (2023), **Self-RAG** embeds reflection tokens directly into the LLM's vocabulary via specialized instruction tuning:

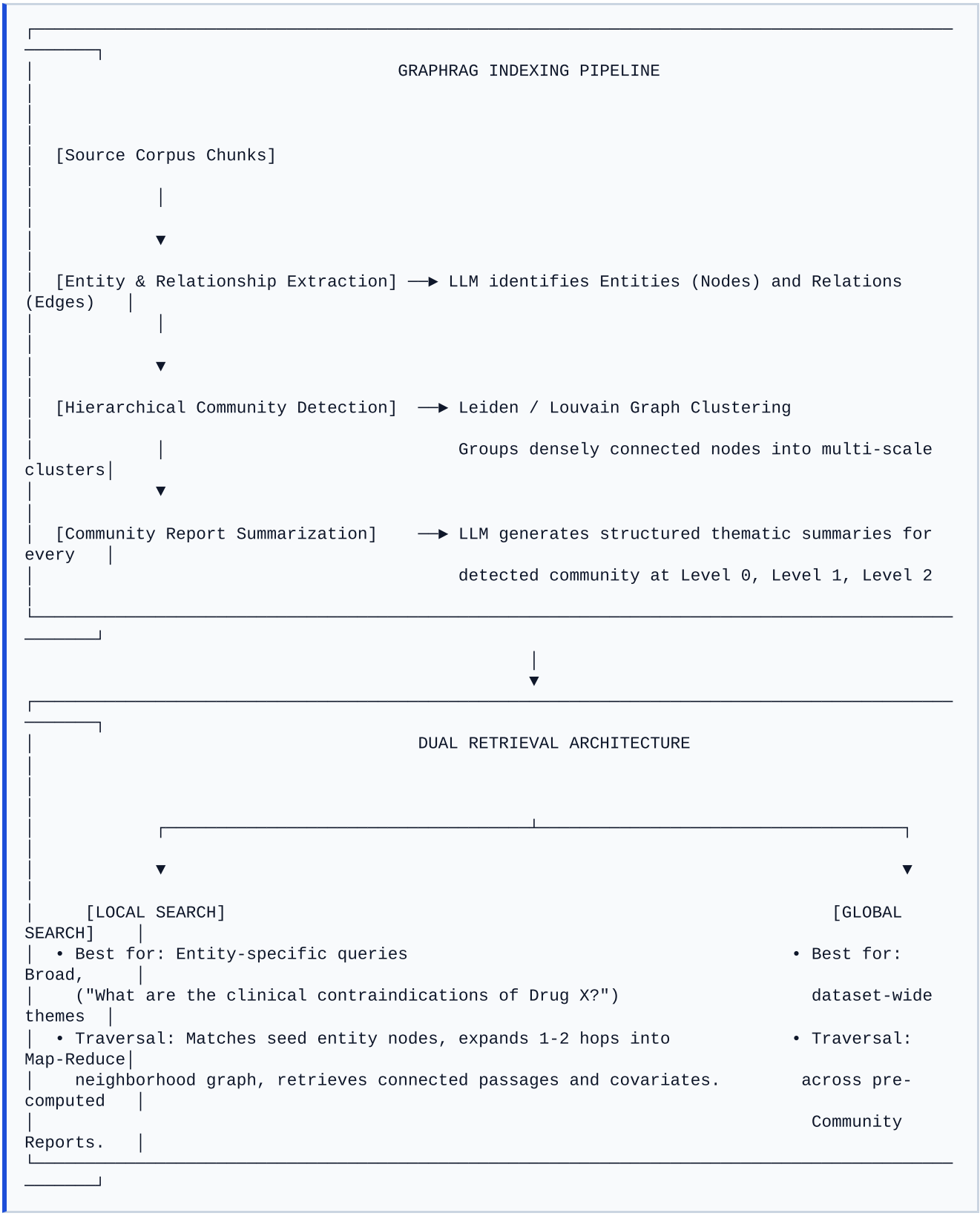
- `[Retrieve]`: Predicts whether external retrieval is even necessary ($P($ `[Retrieve]` $=$ yes$)$ vs. answering directly from parametric memory).
- `[IsRel]` **(Relevance):** Predicts whether a retrieved passage contains relevant evidence to the query.
- `[IsSup]` **(Support / Groundedness):** Evaluates whether the generated sentence is fully supported by the retrieved evidence (hallucination detector).
- `[IsUse]` **(Utility):** Evaluates whether the generated output provides practical utility toward answering the original prompt.

Metric	Corrective RAG (CRAG)	Self-RAG
Intervention Point	Pre-generation middleware (Evaluates documents before generation)	In-generation reasoning (LLM generates tokens conditioned on reflection flags)
Model Requirements	Works out-of-the-box with any standard black-box API (GPT-4o, Claude)	Requires custom fine-tuned weights trained on reflection tokens
Fallback Mechanism	Triggers query rewriting and live web search fallbacks	Prunes unsupported generation beams during decoding
Latency Profile	Moderate (Adds a single fast grading call: ~150–300ms)	High (Requires beam-search sampling over critique tokens)

9.17 GraphRAG & Knowledge-Graph-Augmented Retrieval

Standard vector RAG excels at localized "needle-in-a-haystack" queries where the answer resides in one or two passages. However, it struggles with **cross-document synthesis** and **global thematic summarization** (e.g., *"What are the top five systemic supply-chain vulnerabilities identified across all vendor audit reports?"*). Vector search fails because the answer is distributed across the entire corpus rather than localized in high-similarity chunks.

GraphRAG (Edge et al., Microsoft Research) resolves this by converting the unstructured corpus into a structured **Knowledge Graph (KG)** during indexing.


GRAPHRAG INDEXING PIPELINE
[Source Corpus Chunks]
[Entity & Relationship Extraction] ⟶ LLM identifies Entities (Nodes) and Relations (Edges)
[Hierarchical Community Detection] ⟶ Leiden / Louvain Graph Clustering
Groups densely connected nodes into multi-scale clusters
[Community Report Summarization] ⟶ LLM generates structured thematic summaries for every
detected community at Level 0, Level 1, Level 2
DUAL RETRIEVAL ARCHITECTURE
[LOCAL SEARCH]
• Best for: Entity-specific queries ("What are the clinical contraindications of Drug X?")
• Traversal: Matches seed entity nodes, expands 1-2 hops into neighborhood graph, retrieves connected passages and covariates.
[GLOBAL SEARCH]
• Best for: Broad, dataset-wide themes
• Traversal: Map-Reduce across pre-computed Community Reports.

1. The Indexing Pipeline

1. **Source Text Chunking:** Standard chunking over the corpus.
2. **Domain Entity & Relationship Extraction:** An LLM scans each chunk to extract named entities (type, description) and directed relationships (source, target, relationship description, weight).
3. **Graph Consolidation:** Deduplicates synonym nodes and merges edge weights.
4. **Hierarchical Community Clustering:** The **Leiden Algorithm** partitions the global knowledge graph into hierarchical communities (clusters of densely interconnected entities):
   - *Level 0:* Broad macro-communities (e.g., "North American Aerospace Infrastructure").
   - *Level 1:* Meso-communities (e.g., "Avionics Microcontroller Suppliers").
   - *Level 2:* Micro-communities (e.g., "Specific FPGA Firmware Bugs").
5. **Community Report Synthesis:** An LLM generates a structured report for every community, detailing key entities, recurring dynamics, and unresolved risks.

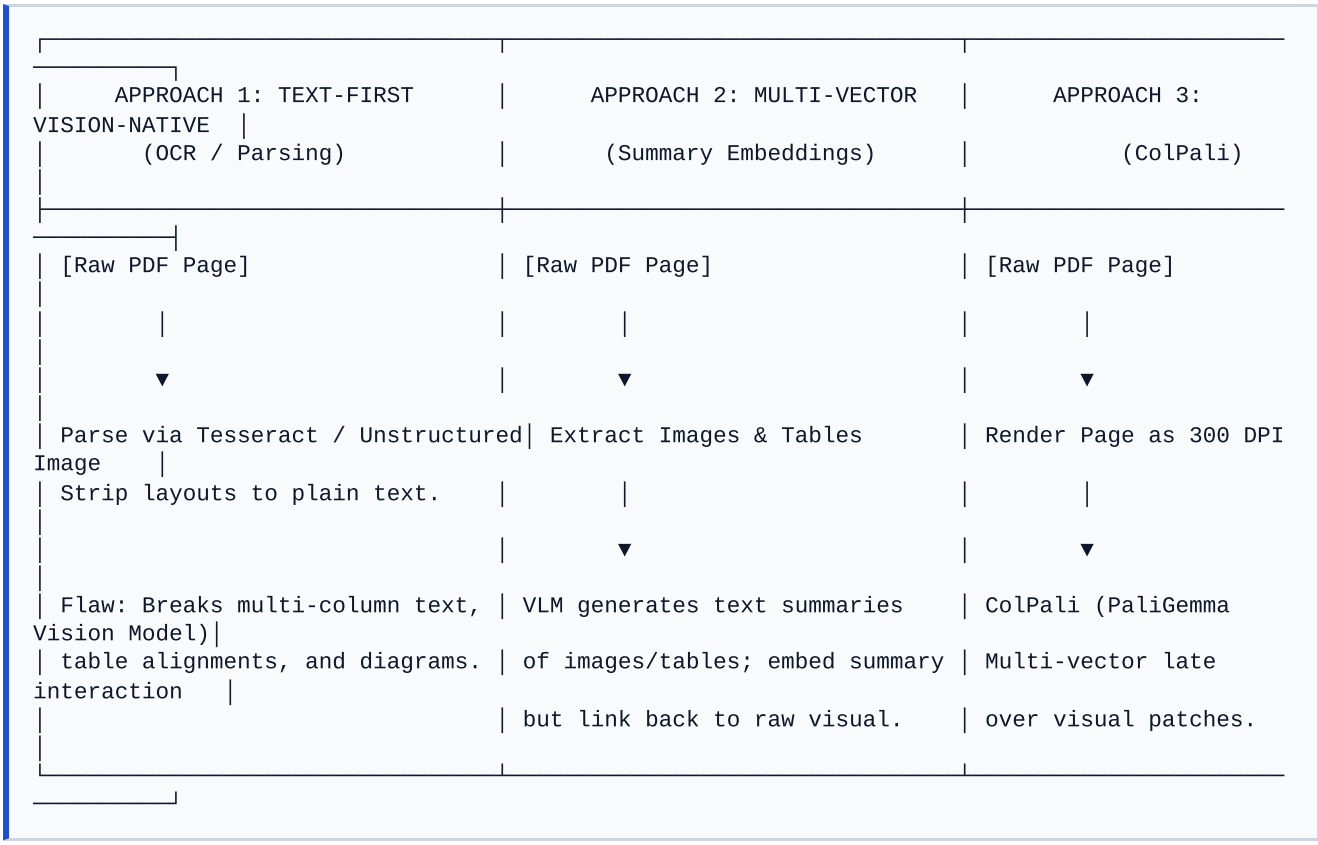
2. Retrieval Modes: Local Search vs. Global Search

- **Local Search (Entity-Focused):**
  - Target: Specific entities and their relational neighborhood.
  - Mechanics: Extracts entities from the user prompt, vector-searches for seed nodes in the graph, traverses incident edges to collect 1-to-2 hop neighbors, and packs the connected entity definitions, relationships, and source text into context.
- **Global Search (Corpus-Wide Thematic Synthesis):**
  - Target: Broad exploratory questions.
  - Mechanics: Uses a **Map-Reduce** pattern across community reports:
    - *Map Phase:* Passes all community summaries at a chosen hierarchical level to the LLM in parallel batches, asking: *"Does this community report answer the user question? If so, generate an intermediate answer with an impact score (0–100)."*
    - *Reduce Phase:* Filters and ranks the intermediate summaries by impact score, feeding them into a final synthesis prompt to generate a comprehensive thematic overview.

Cost Note: While early GraphRAG pipelines incurred high indexing costs, modern frameworks (such as **LazyGraphRAG** and **LightRAG**) use dual-level graph-vector indexing and deferred summarization, bringing indexing costs close to standard vector RAG parity.

9.18 Multimodal RAG: Text, Tables, and Visual Elements

Enterprise documents (financial reports, medical records, engineering schematics) convey information through visual layouts, flowcharts, and embedded tables that degrade when passed through naive text-only OCR pipelines.



1. Architectural Paradigms for Multimodal RAG

1. **Multi-Vector Summary Bridging:**
   - Text blocks, tables, and images are separated during ingestion.
   - Tables are converted to clean Markdown or structured JSON.
   - Images/charts are passed through a Vision-Language Model (VLM, such as GPT-4o or Claude 3.5 Sonnet) with a prompt: *"Provide a detailed factual summary of this chart, including all axes, data points, and legends."*
   - The vector store indexes the **text summary**, but stores the **raw image/table base64 payload** in metadata. When retrieved, the raw visual artifact is passed directly into a multimodal LLM's visual input tokens.
2. **Vision-Native Late-Interaction Retrieval (ColPali):**
   - Eliminates OCR, heuristic bounding-box parsers, and text extractors entirely.
   - Every document page is rendered as an image (e.g., 300 DPI PNG).
   - **ColPali** (based on vision-language models like PaliGemma) encodes image patches directly into multiple vector representations per page, using **ColBERT-style late interaction**:

     $$\text{Score}(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} (\mathbf{q}_i \cdot \mathbf{d}_j)$$

   - The query text interacts with visual patch embeddings directly, retrieving documents based on typography, chart shapes, and visual alignments without losing tabular structure.

9.19 Systematic RAG Failure Modes: Diagnosis and Mitigation

Deploying RAG in production involves navigating three common architectural failure modes:

```
1. RETRIEVAL MISS
   Index fails to fetch the relevant fact.
   Document Corpus: [A][B][C][D] ──► Retrieved: [A, B] (Ground truth was in [C]!)
   Outcome: Generator hallucinates or reports "I don't know."

2. CONTEXT DILUTION
   Retriever returns target fact surrounded by 20 irrelevant noisy passages.
   Prompt Context: [Noise 1][Noise 2][TARGET FACT][Noise 3]...[Noise 19]
   Outcome: Attention heads fail to assign probability mass to the relevant fact.

3. LOST-IN-THE-MIDDLE PHENOMENON
   High LLM retrieval accuracy at boundaries; sharp degradation in the middle.
   Accuracy
     100% ─┐                                                ┌─ 100%
           │                                                │
           └──────┐                                  ┌──────┘
             30%  └──────────────────────────────────┘
                  Context Depth: 0%        50%       100%
```

1. Retrieval Miss (Recall Failures)

- **Root Causes:** Keyword/semantic vocabulary mismatch (user uses slang; document uses formal terminology), out-of-domain embeddings, or inappropriate chunking boundaries that sever a fact across chunk borders.
- **Mitigations:**
  - Implement **Hybrid Search (BM25 + Dense via Reciprocal Rank Fusion)**.
  - Apply **Hypothetical Document Embeddings (HyDE)** to align the query vector with document space.
  - Expand retrieval candidate pools ($K = 50$) followed by Cross-Encoder reranking.

2. Context Dilution & Irrelevant Noise

- **Root Causes:** Stuffing excessive chunks into the context window to maximize recall. High-token prompts reduce the model's focus, increasing the likelihood that irrelevant tokens will distract the attention heads.
- **Mitigations:**
  - Apply Cross-Encoder relevance score cutoffs (e.g., dropping candidates where reranker score $\sigma < 0.70$).
  - Use **Contextual Compression** (e.g., an extractive LLM pass that filters out non-relevant sentences from retrieved chunks prior to final generation).

3. The Lost-in-the-Middle Phenomenon (Liu et al., Stanford)

- **Mechanics:** Decoder-only language models show a U-shaped performance curve when searching for information within their context window:
  - Accuracy is highest for facts placed at the **very beginning** of the prompt context (strong initial token attention bias).
  - Accuracy remains high for facts placed at the **very end** of the context (close to the generation trigger).
  - Accuracy degrades—often dropping below random guessing—when the relevant fact is buried in the **middle 40%–60%** of a large context window.
- **Production Mitigations:**
  - **Order-Aware Context Sorting:** After scoring retrieved chunks with a reranker, sort the top chunks symmetrically:
    - Rank 1 → First position in prompt.
    - Rank 2 → Final position in prompt.
    - Rank 3 → Second position.
    - Rank 4 → Second-to-last position.
    - Lowest-ranked candidates → Middle.

9.20 Production RAG System with Automated RAG Triad Evaluation

Below is a complete, runnable production RAG pipeline built from first principles. It incorporates:

- Hierarchical sentence windowing.
- Query expansion.
- Cross-encoder reranking.
- Grounded generation with source citations.
- **Automated Evaluation Engine** measuring the **RAG Triad** (Context Relevance, Faithfulness/Groundedness, and Answer Relevance).

```
import os
import json
import numpy as np
from typing import List, Dict, Any
from pydantic import BaseModel, Field
from openai import OpenAI

client = OpenAI()

# ==============================================================================
# 1. SAMPLE KNOWLEDGE CORPUS & INGESTION
# ==============================================================================

RAW_CORPUS = [
    {
        "doc_id": "sec_policy_01",
        "title": "Corporate Cryptographic Standards",
        "text": (
            "All production microservices must enforce TLS version 1.3 for data in transit.
"
            "Legacy protocols including SSLv3, TLS 1.0, and TLS 1.1 are explicitly
forbidden. "
            "Symmetric database encryption must utilize AES-256-GCM with automated annual
key rotation managed via AWS KMS. "
            "Any deviation requires an explicit cryptographic waiver signed by the Chief
Information Security Officer (CISO)."
        )
    },
    {
        "doc_id": "dev_ops_02",
        "title": "Production Deployment Runbook",
        "text": (
            "Kubernetes pods must run with read-only root filesystems and drop all Linux
root capabilities. "
            "Database connection pools in FastAPI should default to a maximum of 20
concurrent connections per pod worker. "
            "Deployments must execute zero-downtime rolling updates with maxSurge set to 25%
and maxUnavailable set to 0. "
            "Canary deployments must remain in monitoring for a minimum of 30 minutes before
100% traffic shift."
        )
    }
]

# Simple In-Memory Vector Store
class VectorDocument:
    def __init__(self, doc_id: str, title: str, text: str, embedding: np.ndarray):
        self.doc_id = doc_id
        self.title = title
        self.text = text
        self.embedding = embedding

def get_embedding(text: str) -> np.ndarray:
    resp = client.embeddings.create(input=text, model="text-embedding-3-small")
    vec = np.array(resp.data[0].embedding, dtype=np.float32)
    return vec / np.linalg.norm(vec)  # L2 unit normalization

VECTOR_STORE: List[VectorDocument] = []
for doc in RAW_CORPUS:
    # In production: chunk docs hierarchically. Here we embed clean passages.
    emb = get_embedding(doc["text"])
    VECTOR_STORE.append(VectorDocument(doc["doc_id"], doc["title"], doc["text"], emb))

# ==============================================================================
# 2. RETRIEVAL & RERANKING ENGINE
# ==============================================================================
```

Technical Summary of Module 9 Part 3

Section	Architectural Focus	Primary Failure Mode	Production Best Practice
9.15 Hierarchical Retrieval	Decoupling indexing units from generation units	Embedding dilution on large passages; lost context on tiny chunks	Small-to-Big / Parent-Document indexing via parent UUID pointers.
9.16 Quality-Aware RAG	Self-RAG reflection tokens; CRAG three-way routing	Blindly passing irrelevant retrieved chunks into LLM prompt	Score retrieval confidence γ; route low confidence to live search fallbacks.
9.17 GraphRAG	Knowledge Graph extraction & hierarchical community summarization	Vector RAG failure on broad thematic/dataset-wide queries	Build entity-relationship graph; apply Map-Reduce over community reports.
9.18 Multimodal RAG	Vision-native retrieval (ColPali) & multi-vector summaries	OCR stripping tabular alignments and chart data	Use multi-vector late-interaction over page images or summary bridging.
9.19 Failure Modes	Retrieval misses, context dilution, lost-in-the-middle	Accuracy degradation in middle 50% of context window	Order context symmetrically (top hits placed at start and end).
9.20 Production Project	Grounded pipeline with automated RAG Triad evaluation	Silent production regressions in faithfulness or recall	Track Context Relevance, Faithfulness, and Answer Relevance on every deployment.